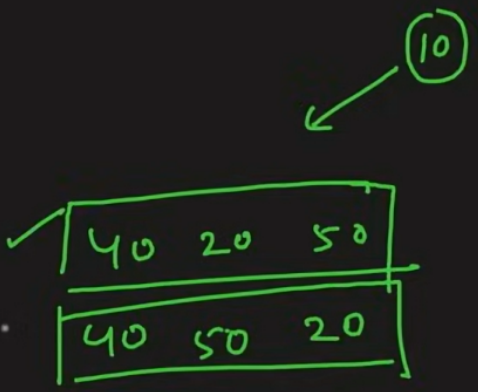
2.30

Construct a Binary Tree from Postorder & Inorder.

Inorder → [40 20 50 | 10 | 60 30]

(Left Root Right)



60 30

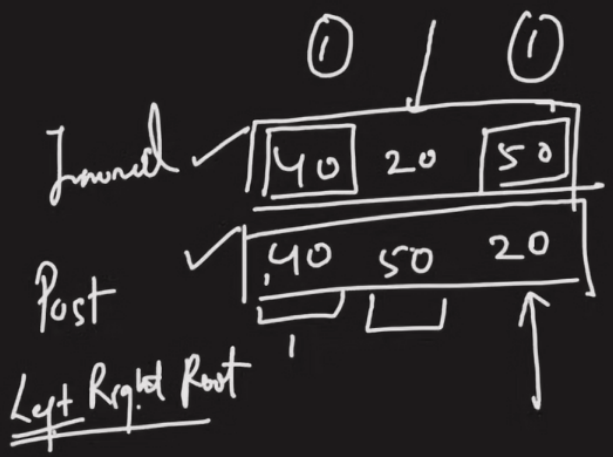
60 30

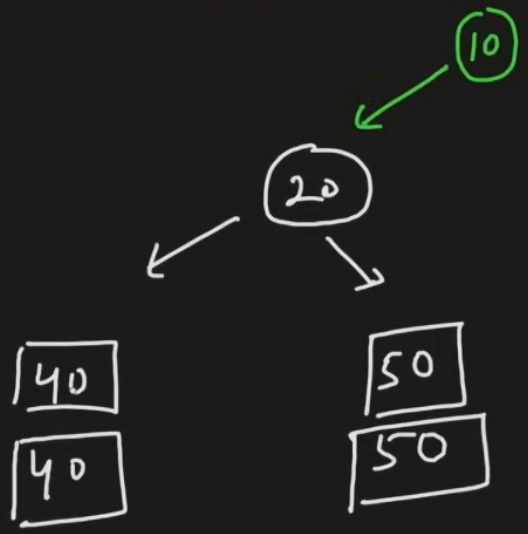
Postorder → [40 50 20 | 60 30 | 10]

(Left Right Root)

2.30

Construct a Binary Tree from Postorder & Inorder.



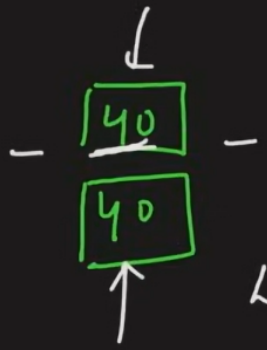


60 30

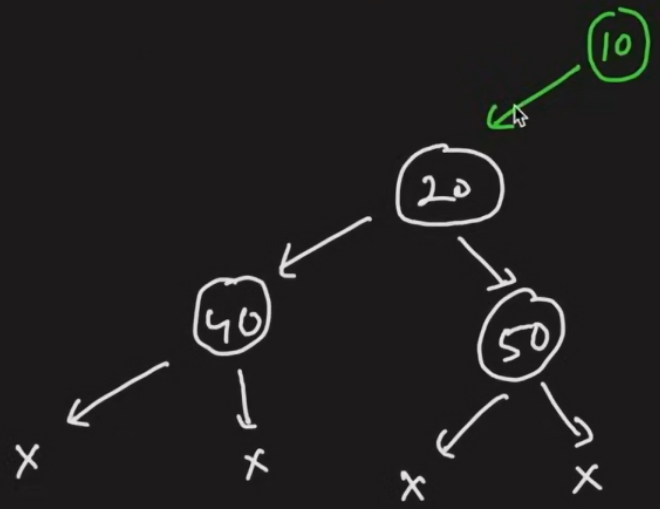
60 30

2.30

Construct a Binary Tree from Postorder & Inorder.



Left Right Root



60 30

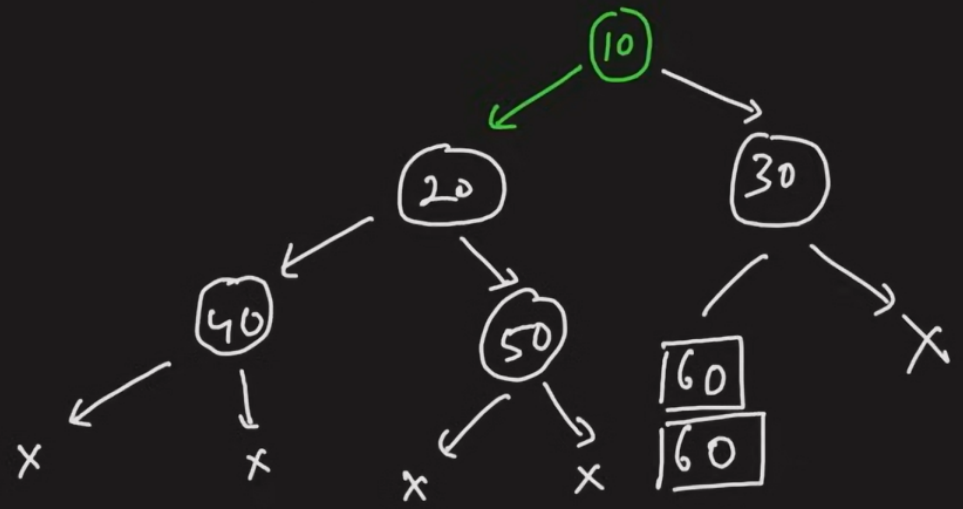
60 30

2.30

Construct a Binary Tree from Postorder & Inorder.

In-> 60 30 _

Post -> 60 30

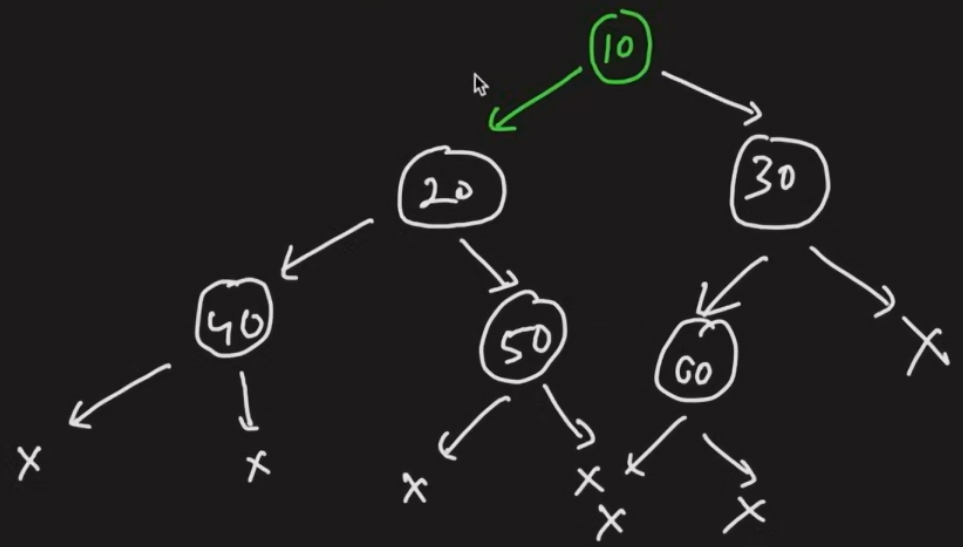


2.30

Construct a Binary Tree from Postorder & Inorder.

In → 40 20 50 10 60 30

Post → 40 50 20 60 30 10



L35. Construct the Binary Tree from Postorder and Inorder Traversal | C++ | Java

C++ • Autocomplete

```cpp
class Solution {
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
        if (inorder.size() != postorder.size())
            return NULL;

        map<int,int> hm;

        for (int i=0;i<inorder.size();++i)
            hm[inorder[i]] = i;

        return buildTreePostIn(inorder, 0, inorder.size()-1, postorder, 0,
                               postorder.size()-1,hm);
    }
    TreeNode* buildTreePostIn(vector<int> &inorder, int is, int ie,
                              vector<int> &postorder, int ps, int pe,
                                    map<int,int> &hm){
        if (ps>pe || is>ie) return NULL;

        TreeNode* root = new TreeNode(postorder[pe]);

        int inRoot = hm[postorder[pe]];
        int numsLeft = inRoot - is;

        root->left = buildTreePostIn(inorder, is, inRoot-1,
                                     postorder, ps, ps + numsLeft -1, hm);

        root->right = buildTreePostIn(inorder,inRoot+1, ie,
                                      postorder, ps + numsLeft, pe-1, hm);
        return root;
    }
};
```

Console • Contribute i ▸ Run Code Submit

Java • Autocomplete

```java
class Solution {
    public TreeNode buildTree(int[] inorder, int[] postorder) {
        if (inorder == null || postorder == null || inorder.length != postorder.length)
            return null;

        HashMap<Integer, Integer> hm = new HashMap<Integer,Integer>();

        for (int i=0;i<inorder.length;++i)
            hm.put(inorder[i], i);

        return buildTreePostIn(inorder, 0, inorder.length-1, postorder, 0,
                               postorder.length-1,hm);
    }

    private TreeNode buildTreePostIn(int[] inorder, int is, int ie,
                                     int[] postorder, int ps, int pe,
                                     HashMap<Integer,Integer> hm){

        if (ps>pe || is>ie) return null;

        TreeNode root = new TreeNode(postorder[pe]);

        int inRoot = hm.get(postorder[pe]);
        int numsLeft = inRoot - is;
        root.left = buildTreePostIn(inorder, is, inRoot-1,
                                    postorder, ps, ps + numsLeft -1, hm);

        root.right = buildTreePostIn(inorder,inRoot+1, ie,
                                     postorder, ps + numsLeft, pe-1, hm);
        return root;
    }
}
```

Console • Contribute i ▸ Run Code Submit

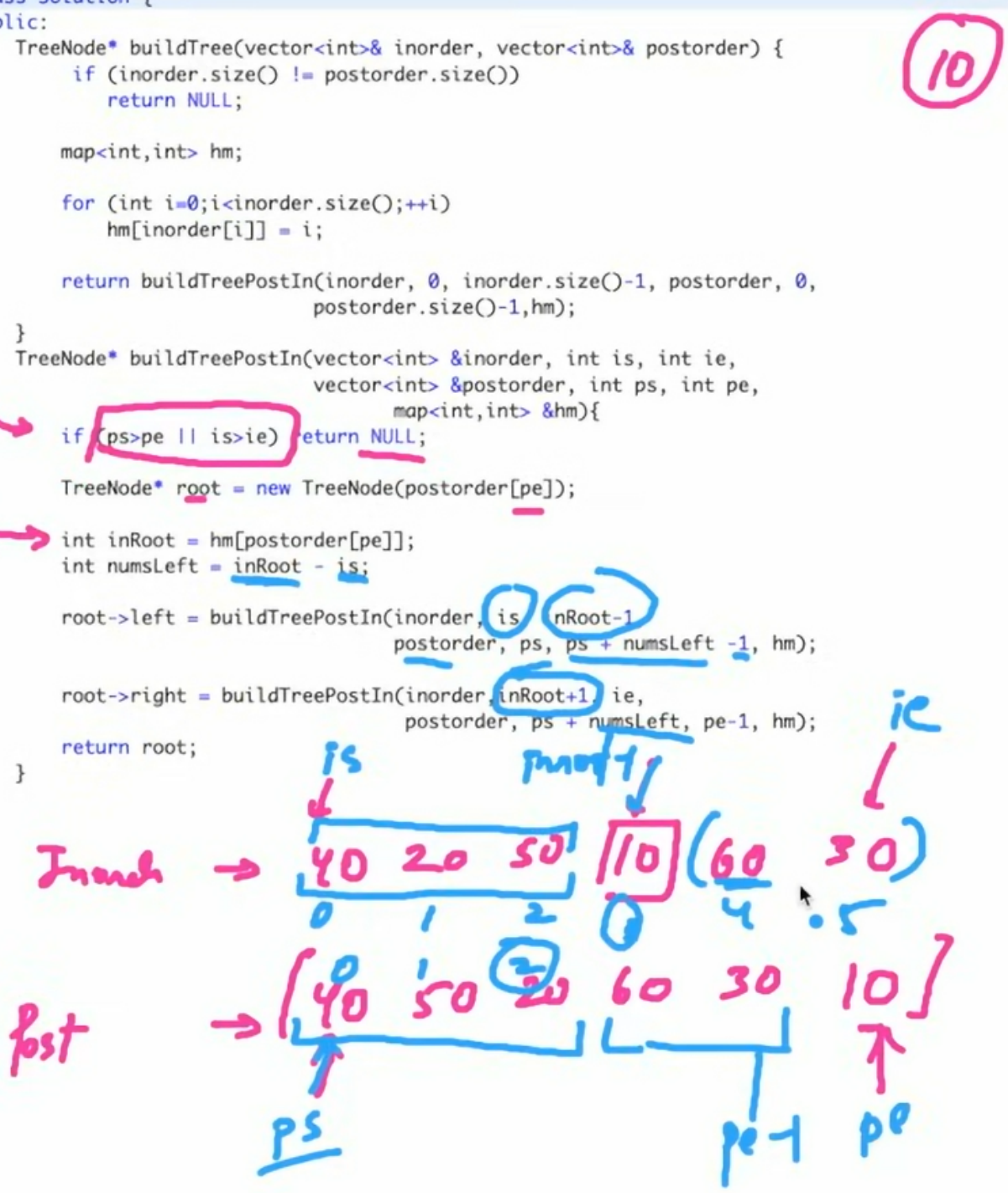
14:07 / 19:45

L35. Construct the Binary Tree from Postorder and Inorder Traversal | C++ | Java

2.30

C++ • Autocomplete

```cpp
class Solution {
public:
    TreeNode* buildTree(vector<int>& inorder, vector<int>& postorder) {
         if (inorder.size() != postorder.size())
            return NULL;

        map<int,int> hm;

        for (int i=0;i<inorder.size();++i)
            hm[inorder[i]] = i;

        return buildTreePostIn(inorder, 0, inorder.size()-1, postorder, 0,
                               postorder.size()-1,hm);
    }
    TreeNode* buildTreePostIn(vector<int> &inorder, int is, int ie,
                              vector<int> &postorder, int ps, int pe,
                                    map<int,int> &hm){
        if (ps>pe || is>ie) return NULL;

        TreeNode* root = new TreeNode(postorder[pe]);

        int inRoot = hm[postorder[pe]];
        int numsLeft = inRoot - is;

        root->left = buildTreePostIn(inorder, is, inRoot-1,
                                     postorder, ps, ps + numsLeft -1, hm);

        root->right = buildTreePostIn(inorder, inRoot+1, ie,
                                      postorder, ps + numsLeft, pe-1, hm);
        return root;
    }
};
```



Console · Contribute i

Run Code · Submit

17:50 / 19:45